Universidad Nacional de Colombia — Sede Manizales

Introducción a las Ciencias de la Computación — Laboratorio computacional

Guía L1 — Primer contacto

¿Cómo se le escribe a alguien que no sabe nada de nosotros?

Laboratorio 1 de 2 · Dos sesiones de 2 horas y una tarea puente · Equipos de 3–4 · Python + IA en Colab

Bitácora del profesor — Dos historias y un mensaje

Hay dos historias de ciencia ficción que no me suelto, y las dos tratan del mismo problema.

En *La llegada* (*Arrival*, 2016), unas naves aterrizan en la Tierra y el ejército le encarga a una lingüista, Louise Banks, que averigüe qué quieren. Los visitantes escriben con círculos de tinta que no se leen de izquierda a derecha ni de derecha a izquierda. Hay una escena que les quiero dejar sembrada: en algún momento la traducción arroja la palabra "arma", y medio planeta se prepara para la guerra. Louise insiste en que la misma palabra podría significar "herramienta". Una mala traducción, hecha con total seguridad, casi acaba el mundo.

En *Project Hail Mary* (la novela de Andy Weir, y la película de este año), un profesor de ciencias se despierta solo en una nave y se encuentra con un ingeniero de otro sistema solar que habla con acordes musicales y cuenta en base seis. No tienen ni una palabra en común. Lo que sí tienen es la física y los números, y desde ahí construyen todo: primero contar, después medir, después palabras, y un diccionario en el computador que crece acorde por acorde.

Lo que me quedó de las dos es una idea que tiene todo que ver con este curso. Cuando dos que no comparten nada quieren hablar, no empiezan por las palabras: empiezan por contar. Uno, dos, tres. Y desde ahí, todo lo demás se pacta: qué es una unidad, qué es un número grande, qué significa un dibujo. Eso que en las películas parece magia de lingüistas tiene en computación un nombre más sobrio: *representación*. Cada vez que ustedes guardan un número, una letra o una foto en un computador, alguien antes que ustedes tuvo que ponerse de acuerdo sobre cómo escribirla con ceros y unos. Este laboratorio es para que ustedes hagan ese acuerdo desde cero, como si nadie lo hubiera hecho antes.

Y resulta que esto no es solo ciencia ficción. El 16 de noviembre de 1974, la humanidad le mandó al espacio un mensaje así: sin idioma, sin instrucciones, solo ceros y unos. Este laboratorio empieza recibiéndolo, como si los extraterrestres fuéramos nosotros, y termina con ustedes escribiendo el suyo.

1. ¿Qué vamos a hacer?

En la sesión 1 van a **recibir** un mensaje real: el que el radiotelescopio de Arecibo (Puerto Rico) envió en 1974 hacia el cúmulo de estrellas M13, a unos 25 000 años luz. Son 1 679 bits, sin una sola instrucción. Lo van a descifrar con herramientas que ustedes mismos programan, y la primera de ellas ya la tienen escrita desde la clase 5. En la sesión 2 van a **responder**: cada equipo será una "especie" distinta, con su propia manera de contar, y le escribirá un mensaje a otra especie del salón, que tendrá que leerlo sin instrucciones mientras el canal le daña bits al azar.

La pregunta que atraviesa las dos sesiones es *¿qué significa representar información?* Al final deberían poder decir, con ejemplos propios, por qué un número, una letra o una medida no "son" ceros y unos, sino que *se acuerdan* como ceros y unos.

Lo real y lo inventado (léanlo dos veces)

- **Real:** el mensaje de Arecibo (1974), su longitud, su contenido y la razón matemática de su longitud. La base seis del ingeniero de *Project Hail Mary* y la escena del "arma" de *La llegada* son ficción, pero el problema que plantean es exactamente el que enfrentaron quienes diseñaron Arecibo.
- **Inventado por nosotros:** las "especies" del salón, sus bases y el canal ruidoso de la sesión 2. Son un modelo de juguete, y como todo modelo hay que poder decir qué deja por fuera.

Logros de este laboratorio

Al terminar este laboratorio estarás en capacidad de:

1. Leer datos crudos en Python y explicar la diferencia entre el *carácter* `'1'` y el *bit* 1.
2. Escribir funciones con contrato (qué reciben, qué devuelven, qué suponen), probarlas con `assert` y usarlas como herramientas sobre datos que no conocían.
3. Convertir números entre bases con división entera y residuo, de dos maneras (con un ciclo y con recursión), y deducir la base de un mensaje ajeno a partir de cómo cuenta.
4. Explicar qué cuesta representar: ancho fijo y desbordamiento, el problema de mandar una cantidad con decimales, y por qué el texto necesita una convención (ASCII, Unicode) que un extraterrestre no tiene.
5. Detectar errores de transmisión con un bit de paridad, y comparar dos maneras

de proteger un mensaje por lo que cuestan y lo que aguantan, midiendo en vez de suponer.

6. Distinguir *reconocer* de *comprender*: decidir, con evidencia, si un asistente de IA está descifrando un mensaje o recitando lo que ya leyó.

Lo que trae de la clase (y dónde lo van a usar)

- **Clase 5:** `esPrimo` con la cota $\sqrt{n}$. Aquí deja de ser un ejercicio y se vuelve la llave del mensaje.
- **Clase 3:** la bisección y la desigualdad $2^k > N$. Aquí responde una pregunta práctica: con cuántos bits se escribe un dígito.
- **Clase 6 y Falsos amigos:** el algoritmo no depende de la notación, la tabla de seguimiento es el juez, y el ejercicio 8 (la máquina misteriosa) saca los dígitos binarios de un número con `mod 2` y `div 2`. Aquí lo van a hacer en cualquier base.
- **La clase de recursión (miércoles 14 de octubre):** $M(n) = M(\lfloor n/10 \rfloor) + n$ mód 10, la suma de las cifras. Cambiar de base es la misma recursión, con otra base y poniendo las cifras en fila en vez de sumarlas.
- **Lo nuevo:** bases distintas de 2 y 10, bits por dígito, ruido y paridad. No se suponen sabidos: se construyen aquí, con las herramientas de arriba.

2. Los datos, los archivos y tres palabras nuevas

Bit es la unidad mínima de información: una respuesta a una pregunta de sí o no. **Codificar** es elegir cómo una información se escribe con bits; **decodificar** es el camino de regreso. Y una **convención** es el acuerdo que hace posible ese regreso: sin ella, los mismos bits pueden significar cualquier cosa.

Un recordatorio sobre bases, porque en este laboratorio las van a usar todo el tiempo. En base 10, $2026 = 2\cdot 1000 + 0\cdot 100 + 2\cdot 10 + 6$: cada posición vale diez veces la de su derecha. En base b, cada posición vale b veces la de su derecha, y los dígitos van de 0 a $b-1$. Así, en base 2, $1011_2 = 8+2+1 = 11$, y en base 6, $13050_6 = 1\cdot 1296 + 3\cdot 216 + 0\cdot 36 + 5\cdot 6 + 0 = 1974$. Para pasar de un número a sus dígitos se divide una y otra vez: el residuo es la última cifra y el cociente es lo que falta por escribir. Es la máquina misteriosa, en cualquier base.

Los archivos del laboratorio

Todo está en el repositorio de laboratorios del curso, `github.com/cmorregof/labs-calculo-2026-2`, en la carpeta `introduccion_cc`. Ábranlos en Colab desde los enlaces del `README`.

- `L1_sesion1.ipynb` — Recibir: descifrar Arecibo.
- `L1_tarea_puente.ipynb` — Reproducir el dato, cambiar de base de dos maneras y diseñar su especie.
- `L1_sesion2.ipynb` — Responder: escribir, proteger y mandar el mensaje de su especie; leer uno ajeno.
- `congelar_arecibo.py` — El programa con el que se congeló el mensaje. En la tarea puente lo corren ustedes.
- `datos/arecibo.txt` (en la carpeta `datos` de la raíz del repositorio) — El mensaje congelado: una sola línea de 1 679 caracteres `0` o `1`.
- `bitacora_IA_L1.md` — La plantilla de la bitácora individual.

Los cuadernos ya traen la estructura: cada función con su contrato escrito, un espacio `# TU CÓDIGO AQUÍ`, y debajo una celda de pruebas con `assert`. Si una prueba falla, la celda se queja con un `AssertionError`: no es un regaño, es la máquina diciéndoles dónde mirar. Las celdas marcadas como CAJA ya vienen hechas; basta con entender su contrato, igual que con `print` o `range`.

Fíjense en una ironía que vamos a discutir: el archivo guarda cada bit del mensaje como un *carácter*, y cada carácter ocupa 8 bits en el disco. El mensaje de 1 679 bits pesa en su computador 13 432 bits. Representar algo siempre tiene un costo, y ese costo depende de la convención.

3. Sesión 1 — Recibir: el mensaje de Arecibo

Momento 1 — Predecir (a mano, antes de abrir el cuaderno)

En papel, 15 minutos, foto al cuaderno:

P1. Les llega una cadena de 1 679 ceros y unos, sin nada más. ¿Qué es lo primero que intentarían para encontrarle sentido? Escriban tres ideas, en orden.

P2. Si alguien quisiera mandar un *dibujo* con bits, ¿cómo lo haría? ¿Qué tendría que adivinar quien lo recibe?

P3. ¿De cuántas maneras se pueden acomodar 1 679 cuadritos en un rectángulo completo, sin que sobre ni falte ninguno? Respondan sin computador, con lo que saben de divisores.

Momento 2 — Ejecutar. En `L1_sesion1.ipynb`, con el monitor. Cada herramienta es una función, y de cada función se escribe primero el contrato y después el código.

1. **Traer y conocer el dato.** La caja `cargar_mensaje()` trae el mensaje, y `huella(bits)` les da su número de unos y su SHA-256, una "huella digital" que cambia por completo si cambia un solo bit (con la misma idea su banco sabe que nadie alteró una transferencia en el camino). Antes de calcular nada, contesten con código: ¿cuántos caracteres?, ¿cuáles distintos?, ¿cuántos unos y cuántos ceros? Un dato que no se miró antes de usarlo es un dato en el que se confió sin razón.

2. **La llave: `es_primo(n)`.** Traduzcan a Python el `esPrimo` de la clase 5 como función que *devuelve* `True` o `False`. Pruébenla con 91, 97 y 1 679.

3. **Las formas posibles: `rectangulos(n)`.** La lista de todas las parejas (filas, columnas) con filas × columnas = n. Córranla con 1 679 y confronten P3. La pregunta de fondo, para el informe: *¿por qué quien diseñó el mensaje eligió un número que es producto de dos primos?* Comparen con 1 680, que se puede acomodar de 40 maneras. Quien termine antes mejora el algoritmo: probar solo hasta $\sqrt{n}$, cada divisor pequeño trae de regalo a su pareja, y se cuentan las vueltas de las dos versiones.

4. **Ver: `filas_de` y `dibujar`.** Una función que *devuelve* los renglones (y por eso se puede probar) y otra que los *imprime* con un bloque lleno para el 1 y un espacio para el 0. Prueben con 73 columnas y con 23. Una de las dos es ruido; la otra no. Este es el momento de dejarlo correr y esperar a ver si coincide con lo que pensaban. Una caja, `dibujar_imagen`, lo muestra como imagen con la cuadrícula numerada.

5. **Leer: `leer_numero` y `columna`.** Arriba del dibujo hay números en binario, escritos en vertical. `leer_numero` convierte una lista de bits en un entero con un acumulador (duplicar lo que llevo y sumarle el bit: con `1011`, $0 \to 1 \to 2 \to 5 \to 11$), y `columna` saca un pedazo de columna de la cuadrícula. Hay una sola dirección de lectura que hace que la primera sección diga algo que reconocen; encuéntrenla probando las dos y justifiquen cuál es. Después lean la segunda sección: son cinco números, y en la tabla periódica van a reconocer a quiénes corresponden.

6. **Inventario.** Con el dibujo y sus funciones, la tabla de lo que creen que dice cada parte del mensaje, separando lo que descifraron *con código* de lo que están adivinando *con los ojos*.

Momento 3 — Explicar. Media página por equipo. La parte obligatoria: la **lista de convenciones** que el emisor supuso que el receptor ya conocía (que los bits forman un rectángulo, en qué orden se leen, que los números van en base dos, que existe una tabla periódica…). Para cada una, digan si un extraterrestre de verdad podría deducirla o si la tendría que adivinar. Y una frase que empiece por "Representar es…", escrita con sus palabras.

4. Tarea puente (entre las dos sesiones): no me creas — y diseñen su especie

En `L1_tarea_puente.ipynb`. Las tres partes son del equipo, y la C tiene que estar hecha antes de la sesión 2: allá no hay tiempo de diseñar.

Parte A, reproducibilidad. El archivo `datos/arecibo.txt` lo congelé yo con `congelar_arecibo.py`. ¿Y por qué habrían de creerme? En ciencia la confianza no se pide: se *reproduce*. Descarguen el programa y córranlo ustedes como se corre en una terminal (en Colab, una línea que empieza con `!` es una orden para la terminal del computador de Google). El programa trae el mensaje de su fuente original y escribe su propia copia. Comparen *su* huella con las cifras de control del `README` de `introduccion_cc`: el número de unos y el SHA-256 tienen que coincidir exactamente. Si algo no cuadra, no está "casi bien": hay una diferencia, y encontrarla es la tarea. Y cinco líneas: ¿por qué comparar dos huellas de 64 caracteres es una manera confiable de saber que dos archivos de 1 679 caracteres son idénticos?

Parte B, dos maneras de cambiar de base. Escriban `a_base_ciclo(n, b)` con un `while` (la máquina misteriosa, con b en lugar de 2) y `a_base_rec(n, b)` recursiva: los dígitos de n en base b son los dígitos de $n \operatorname{div} b$ seguidos del dígito n mód b. ¿Cuál es el caso base? Cuidado con los falsos amigos de la clase de recursión: la llamada tiene que achicarse y tiene que llegar al caso base. Escriban también la inversa, `de_base(digitos, b)`, que es su `leer_numero` con b en lugar de 2. El cuaderno compara las dos maneras en mil casos al azar. Hagan a mano la tabla de seguimiento de `a_base_rec(1974, 6)`, con la pila: ¿qué queda esperando en cada llamada?

Parte C, su especie. El monitor lanza un dado y le asigna a cada equipo una base: 2, 3, 5, 6, 8 o 12. Esa es su manera de contar, y todos los números de su mensaje irán en esa base. En papel, diseñen lo que le van a decir a otra especie, en tres o cuatro secciones. La primera es obligatoria: *contar*, 1, 2, 3, … en su base. Las demás son suyas: quiénes son, dónde viven, algo que les importe. Y escriban el **contrato de lectura**: la lista de pasos que el receptor tendría que deducir, en orden, para entenderlos. Ese contrato es su predicción para la sesión 2, y no se lo muestran a nadie.

5. Sesión 2 — Responder: el mensaje de su especie

Momento 1 — Predecir

P4. ¿Qué fracción de su mensaje creen que la otra especie va a entender? Un número entre 0 y 1, y qué sección creen que se va a perder primero.

P5. Si el canal daña 1 de cada 100 bits, ¿cuántos bits dañados esperan en su mensaje? ¿Y si lo mandan tres veces?

Momento 2 — Ejecutar. En `L1_sesion2.ipynb`:

1. **Sus herramientas.** Peguen sus funciones de la sesión 1 y de la tarea puente. Son suyas: se reutilizan, no se reescriben.

2. **Bits por dígito.** Si cuentan en base 6, sus dígitos van de 0 a 5. ¿Con cuántos bits se escribe cada uno? Es la desigualdad de la clase 3: el menor k con $2^k \geq b$. Escriban `bits_por_digito(b)` y `digito_a_bits(d, ancho)`, que escribe un dígito con exactamente `ancho` bits.

3. **El mensaje, por secciones.** Un mensaje es una lista de renglones de `COLUMNAS` bits, y cada sección es una función que devuelve renglones. La obligatoria es `codificar_conteo`: los números del 1 en adelante, en su base, uno por renglón. Quien sepa leer ese conteo descubre su base, porque ve en qué momento se acaban los dígitos y aparece un "10". Después vienen sus secciones, cada una con contrato, y `ensamblar`, que las pone una debajo de otra. Si mañana quieren cambiar una sección, cambian una función y nada más: eso es pensar en módulos. Y la forma: ¿su mensaje tiene una longitud que obligue a una sola manera de dibujarlo, como el de Arecibo?

4. **Las preguntas incómodas de la representación.** Su mensaje tiene que responder al menos dos de estas, y el cuaderno tiene que explicar cómo lo resolvieron:

   - *Ancho fijo.* Si cada número va en un renglón, ¿cuál es el número más grande que cabe? ¿Qué pasa si quieren decir cuántos habitantes tiene Manizales? (Desbordamiento.)
   - *Decimales.* ¿Cómo dicen la altura de Manizales en metros si el otro no sabe qué es un metro, ni dónde va la coma? (Pista real: las placas de las sondas Pioneer, en 1972, usaron una propiedad del átomo de hidrógeno como regla para medir, porque el hidrógeno es igual en todo el universo.)
   - *Texto.* ¿Por qué no pueden mandar la palabra "Manizales" en ASCII o en Unicode? ¿Qué sí pueden mandar?
   - *Forma.* ¿Cómo va a saber la otra especie cuántas columnas tiene su mensaje?

5. **El canal.** `ruido(bits, p)` voltea cada bit con probabilidad p. Dos maneras de protegerse: *paridad* (`con_paridad` agrega a cada renglón un bit para que su número de unos sea par, y `filas_danadas` dice cuáles llegaron impares) y *repetición* (`repetir3` manda cada bit tres veces y `votar` se queda con el que más aparece). Midan, no supongan: corran el canal mil veces con $p = 0{,}01$ y cuenten cuántas transmisiones tuvieron error, cuántas detectó la paridad, cuántas se le escaparon (¿cómo puede pasar?), cuántas recuperó exactas la repetición, y cuánto cuesta cada una en bits.

6. **Mandar.** Elijan su protección, pasen el mensaje una sola vez por el canal y guárdenlo en `mensaje.txt`: una sola línea de bits, sin decir cuántas columnas tiene ni cómo se protegió. Se lo entregan al monitor.

7. **El día del contacto.** El monitor reparte los mensajes entre especies. Cada equipo recibe uno ajeno, sin el contrato de lectura, y tiene 30 minutos para descifrarlo *con las funciones de la sesión 1*. Si `rectangulos`, `dibujar`, `columna` y `de_base` sirven para un mensaje que nunca habían visto, eran herramientas de verdad; si solo servían para Arecibo, eran un truco.

8. **La revelación.** Cada especie lee en voz alta lo que entendió del mensaje recibido, y la especie emisora revela su contrato de lectura. Anoten qué se entendió, qué se perdió y qué se entendió *mal* (recuerden la palabra "arma").

Momento 3 — Explicar. Una página: confronten P4 con lo que de verdad se entendió de su mensaje; identifiquen la convención que supusieron *sin darse cuenta* y que la otra especie no tenía; expliquen con sus mediciones cuál de las dos protecciones elegirían para un mensaje que tarda 25 000 años en llegar, pensando en la diferencia entre *detectar* un error y *corregirlo*; y vuelvan a escribir la frase "Representar es…". ¿Cambió desde la sesión 1?

Protocolo de IA

La IA es herramienta y es objeto de estudio en este laboratorio, y aquí las dos cosas se cruzan como en ningún otro. Las tres reglas del curso aplican (intento propio antes de consultar; la IA no es la autoridad final; todo uso va a la bitácora individual: el prompt exacto, qué salió, qué se aceptó, corrigió o rechazó, y cómo se comprobó). La plantilla está en `bitacora_IA_L1.md`.

- *El experimento estrella (sesión 1):* denle al asistente la cadena de 1 679 bits sin decirle qué es, y pídanle que la descifre. Guarden la respuesta. ¿La *descifró* (hizo cuentas que ustedes pueden verificar con sus funciones) o la *reconoció* (sabía de antemano qué era y recitó)? ¿Cómo lo distinguen? Propongan una prueba.
- *La contraprueba (sesión 2):* denle el mensaje de otra especie del salón, que no está en ningún lugar de internet: no lo puede reconocer, solo lo puede descifrar. Comparen lo que dice el asistente con lo que reveló la especie emisora. ¿Tradujo

con seguridad algo que no estaba ahí? Esa es la escena del "arma", y la acaban de medir.

- *Permitido:* preguntarle qué es la paridad, cómo se lee un archivo en Python, qué significa un mensaje de error. Pedirle ejemplos distintos a los de esta guía. Pedirle que les explique por qué falla una prueba, sin darles el código corregido.
- *Prohibido:* pedirle las funciones de los cuadernos, el contrato de lectura, el diseño del mensaje, la lista de convenciones o los momentos 3. Todo integrante debe poder explicar cualquier celda si el monitor se la señala.

6. La rúbrica de comprensión computacional

El curso califica la comprensión con el Modelo de Bloques de Schulte: no basta con que el programa corra; hay que entender lo que *dice* (texto), lo que *hace* cuando corre (ejecución) y *para qué sirve* (propósito), a distintas escalas. Este es el mapa de dónde se mira cada cosa en este laboratorio:

	Texto (qué dice)	**Ejecución** (qué hace al correr)	**Propósito** (para qué sirve)
Instrucción	qué hace `n % b` y `n // b`	la pila de `a_base_rec(1974, 6)`	por qué el residuo es un dígito
Bloque	el cuerpo de `leer_numero`	qué pasa al leer al revés	el contrato de cada función
Relaciones	cómo `ensamblar` usa las secciones	el mensaje pasando por el canal	por qué las herramientas de la sesión 1 sirven en la 2
Programa	el cuaderno completo	la reproducibilidad de la tarea puente	qué convenciones supone el mensaje entero

Entregables y rúbrica

Entregables (cierre de la semana de la sesión 2): (1) los tres cuadernos del equipo (`L1_sesion1`, `L1_tarea_puente`, `L1_sesion2`), ejecutados de arriba a abajo sin errores, con las fotos de las predicciones y los tres momentos completos, guardados en el repositorio `intro-cc` de cada integrante (*Archivo → Guardar una copia en GitHub*); (2) en el mismo repositorio, `mensaje.txt` con los bits que mandó su especie; (3) bi-

tácora de IA individual (1 página) con los dos experimentos; (4) en el cuaderno de la tarea puente: la verificación reproducida, `a_base` de las dos maneras con la tabla de la pila, y el contrato de lectura escrito *antes* de la sesión 2.
Rúbrica (sobre 5.0): funciones con contrato, pruebas que pasan y uso sobre un mensaje ajeno (1.25) · lectura de Arecibo justificada con código, incluida la razón de los dos primos (1.0) · cambio de base de dos maneras, con la tabla de la pila y el caso base explicado (0.75) · el mensaje propio y dos preguntas incómodas de la representación resueltas (1.0) · paridad contra repetición, medido y no supuesto (0.5) · comunicación, lista de convenciones y confrontación de predicciones (0.5). Cuaderno que no ejecuta completo: se devuelve sin calificar.

7. Para ir más lejos (opcional)

Corregir, no solo detectar. En 1950, Richard Hamming, cansado de que la máquina de los laboratorios Bell detectara errores durante el fin de semana y se detuviera sin poder arreglarlos, inventó una manera de agregar bits que no solo dice *que* hubo un error, sino *dónde*. Con el código de Hamming (7,4) se mandan 4 bits de información en 7, y cualquier bit volteado se corrige solo. Impleméntenlo, pásenlo por su canal y compárenlo con la paridad y la repetición: ¿cuántos bits cuesta?, ¿cuántos errores aguanta?

Un idioma desde cero. En 1960, el matemático Hans Freudenthal diseñó *Lincos*, un idioma para hablar con extraterrestres que empieza enseñando aritmética y construye, sobre ella, la lógica y después las palabras. Es el mismo camino del ingeniero de *Hail Mary*, escrito como libro. ¿Hasta dónde llegarían ustedes con su base y sus bits?